AI Clinic Voice Receptionist – Build & Deployment Guide

Client location: Calgary, Alberta, Canada. Development and testing: India. Production deployment: Canada.

Model: one dedicated agent per clinic (single-tenant). Shared core codebase, separate config and deployment per clinic.

> All prices are approximate and in USD. Vendor prices change often, so check each vendor's pricing page before you quote a client.

1. What the system does

A patient calls the clinic's Calgary phone number. The AI answers, understands what they say, replies in a natural voice and can:

- Answer FAQs (hours, location, services, parking, insurance)
- Book, reschedule or cancel appointments in the clinic's booking system
- Take messages and send them to the front desk
- Transfer urgent or clinical calls to a human, or tell callers to dial 911 in an emergency

2. Technology stack

Layer	Recommended	Alternatives	Role
Phone number / carrier	Twilio	Telnyx	Calgary number (403/587/825), carries the call
Call bridge	SIP trunk	—	Connects the phone network to LiveKit
Real-time media server	LiveKit Cloud or self-hosted LiveKit	Pipecat	Streams the audio between the caller and the agent
Agent framework	LiveKit Agents (Python)	Pipecat	Your code: the logic, turn-taking, tools

Layer	Recommended	Alternatives	Role
Speech-to-text (STT)	Deepgram (Nova)	AssemblyAI, Whisper (self-hosted)	Turns the caller's speech into text
LLM (the thinking)	Qwen 2.5 / Qwen 3 (7B–14B) or GLM	GPT-4o-mini, Claude Haiku	Decides what to say and which tool to call
Text-to-speech (TTS)	Cartesia	ElevenLabs	Turns the reply into a voice
Voice activity detection	Silero VAD (built into LiveKit)	—	Detects when the caller starts and stops talking
Backend / tools	FastAPI (Python)	Node.js	Booking API calls, messages, logging
Database	PostgreSQL	SQLite for testing	Call logs, messages, settings
Hosting (production)	AWS ca-west-1 (Calgary)	AWS ca-central-1 (Montreal), Azure/GCP Canada	Keeps patient data in Canada
Booking system	Jane App (common in Calgary)	Accuro, Oscar, Cliniko	Appointments

3. Architecture: how each part talks to the next

Call flow, step by step:

1. **Patient → Twilio:** the patient dials the clinic's Calgary number. Twilio receives the call on the regular phone network (PSTN).
2. **Twilio → LiveKit (SIP trunk):** Twilio forwards the call over SIP to your LiveKit SIP endpoint. LiveKit creates a room and adds the caller as a participant.
3. **LiveKit → your agent:** LiveKit dispatches your Python agent worker into that room. Audio streams both ways over WebRTC.
4. **Agent → Deepgram (STT):** the agent streams the caller's audio to Deepgram over a WebSocket and gets live text back.
5. **Agent → LLM:** the text, plus the conversation history and the clinic's system prompt, goes to the LLM through an OpenAI-compatible API (vLLM, Groq, OpenRouter or OpenAI).

6. **LLM → tools (function calling):** if the LLM decides to book or check a slot, it outputs a tool call. Your agent runs the function, which calls your FastAPI backend, which calls the Jane or clinic API.
7. **LLM → Agent → Cartesia/ElevenLabs (TTS):** the reply text is streamed to the TTS over a WebSocket. Audio comes back in chunks.
8. **Agent → LiveKit → Twilio → patient:** the audio plays to the caller. The target is under about 1 second of total response time.
9. **After the call:** the transcript, summary and any messages are saved to PostgreSQL and emailed or texted to the clinic.

Simple view:

Patient phone → Twilio (PSTN) → SIP trunk → LiveKit room → Python agent → [Deepgram STT → LLM (Qwen) → Cartesia TTS] → back through LiveKit → Twilio → patient

Side channel: LLM tool call → FastAPI backend → Jane App API / email / SMS / database

Interfaces summary:

From	To	Protocol
Phone network	Twilio	PSTN
Twilio	LiveKit	SIP (Elastic SIP Trunk)
LiveKit	Agent	WebRTC / LiveKit SDK
Agent	Deepgram	WebSocket streaming
Agent	LLM	HTTPS, OpenAI-compatible chat completions (streaming)
Agent	TTS	WebSocket streaming
Agent	Backend	HTTPS REST (JSON)
Backend	Booking system	REST API or webhook
Backend	Clinic staff	Email (SMTP/SendGrid) or SMS (Twilio)

4. Single-tenant setup (one agent per clinic)

- Keep **one core Git repository** with the agent code.

- Each clinic gets its own **config file** (clinic.yaml): name, hours, address, FAQs, services, transfer number, booking API keys, voice choice, greeting.
- Each clinic gets its **own deployment**: a separate server or container, a separate Twilio number, a separate LiveKit project or dispatch rule, and a separate database.
- Bug fixes go into the core repo once, then you redeploy each clinic.
- Upside: full data isolation per clinic, which suits health privacy rules.

Example folder layout:

- core/ (agent.py, tools.py, prompts.py)
- clinics/clinic_a/config.yaml
- clinics/clinic_b/config.yaml
- deploy/ (Dockerfile, docker-compose.yml)

5. Step-by-step build procedure

Step 1 – Accounts

1. Twilio: sign up and use the trial credit.
2. LiveKit Cloud: free tier, or self-host LiveKit later.
3. Deepgram: sign up (it usually comes with about $200 free credit).
4. Cartesia or ElevenLabs: free tier for testing.
5. LLM: a Groq, OpenRouter or Together account for hosted Qwen or GLM, or a GPU server for self-hosting.

Step 2 – Local agent (in India)

1. Install Python 3.10+.
2. Run `pip install "livekit-agents[deepgram,openai,cartesia,silero]"`.
3. Put the API keys in a .env file: LIVEKIT_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, DEEPGRAM_API_KEY, CARTESIA_API_KEY, and LLM_BASE_URL plus LLM_API_KEY.
4. Write the agent using AgentSession with: STT = Deepgram, LLM = the OpenAI plugin pointed at your Qwen endpoint (base_url), TTS = Cartesia, VAD = Silero.
5. Test it in the browser with the LiveKit Agents Playground. No phone number is needed yet, so this is free.

Step 3 – Using Qwen / GLM

- **Hosted (testing):** point the OpenAI plugin's base_url at Groq, OpenRouter or Together and choose a Qwen model.
- **Self-hosted (production):** run `vllm serve Qwen/Qwen2.5-14B-Instruct` on a GPU, which gives you an OpenAI-compatible endpoint at http://your-server:8000/v1.
- Use 7B–14B models for speed. Bigger models add latency on calls.
- Check the license of the exact model version. Most Qwen models are Apache 2.0.

Step 4 – Tools (function calling)

Define functions the LLM can call:

- check_availability(date, service)
- book_appointment(name, phone, date, time, service)
- cancel_appointment(phone, date)
- take_message(name, phone, reason)
- transfer_to_human()

Each one calls your FastAPI backend, which talks to the booking system.

Step 5 – Booking system integration

1. Ask the clinic which software they use.
2. Jane App: ask Jane about API or partner access (it's limited, so confirm early).
3. If there's no API: the agent takes the booking request and sends it to staff by email or SMS for confirmation, or it uses a shared Google Calendar.

Step 6 – Connect a real phone number

1. In Twilio, buy a Calgary number (403, 587 or 825).
2. Create an Elastic SIP Trunk in Twilio and set its origination URI to your LiveKit SIP URI.
3. Attach the number to the trunk.
4. In LiveKit, create an inbound SIP trunk and a dispatch rule that sends calls into a room with your agent.
5. Call the number and the agent should answer.

Step 7 – Testing from India

- Use the LiveKit Playground (browser) for most tests. It's free.
- For real phone tests, call the Calgary number using Twilio Voice from the browser or a VoIP app (Zoiper or similar with a SIP account) so you avoid international call charges.
- Use **fake patient data only**.
- Test: different accents, background noise, interruptions, silence, emergencies, wrong numbers, and booking edge cases.
- Measure latency, aiming for under 1 to 1.5 seconds per reply.

Step 8 – Deploy to Canada

1. Containerise the agent and backend with Docker.
2. Deploy to AWS ca-west-1 (Calgary) or ca-central-1 (Montreal): EC2, ECS or Lightsail for the agent, RDS PostgreSQL for the database, and a GPU instance if you self-host Qwen.
3. Self-host LiveKit in Canada too, or confirm LiveKit Cloud's data region.
4. Use HTTPS everywhere, encrypt the database, and keep secrets in AWS Secrets Manager.
5. Set up monitoring (CloudWatch) and alerts.
6. Do a soft launch: after-hours calls first, then overflow calls, then all calls.

6. Privacy and compliance (Alberta)

- **Alberta Health Information Act (HIA)** applies to health information handled by custodians such as clinics. The clinic is the custodian; you are their "affiliate" or service provider.
- **PIPEDA / PIPA (Alberta)** may also apply.
- Keep patient data in Canada wherever possible. Self-hosting STT, LLM and TTS in Canada is the strongest option. If you use US vendors (Deepgram, Cartesia, ElevenLabs, OpenAI), the clinic may need to do a Privacy Impact Assessment and get data-processing agreements, and they should turn on zero data retention where it's offered.
- Tell callers at the start that they're talking to an AI and that the call may be recorded.
- The agent must never give medical advice. Emergencies go to 911, and clinical questions go to staff.

- Sign a written agreement with the clinic covering data handling.
- I'm not a lawyer, so have the clinic confirm compliance with their privacy officer or legal adviser.

7. Costs

Testing phase (India)

Item	Cost
LiveKit Cloud	Free tier
Deepgram	About $200 free credit
Cartesia / ElevenLabs	Free tier, or about $5/month
Hosted Qwen/GLM (Groq/OpenRouter)	Free tier or a few cents
Twilio	Trial credit (about $15)
Total	**About $0–20**

Production (per clinic, per month, approximately)

Item	Approximate price	For 1,000 call minutes/month
Twilio Calgary number	$1–2/month	$1–2
Twilio inbound voice / SIP	about $0.005–0.01/min	$5–10
LiveKit Cloud (or self-host)	about $0.01/min, or the server cost	$10
Deepgram STT	about $0.004–0.008/min	$4–8
Cartesia / ElevenLabs TTS	about $0.02–0.08/min	$20–80
LLM, hosted Qwen	fractions of a cent per call	$1–5
LLM, self-hosted Qwen on a GPU	$300–800/month per GPU server	$300–800

Item	Approximate price	For 1,000 call minutes/month
Agent + backend server (AWS)	$20–60/month	$20–60
Database (RDS small)	$15–30/month	$15–30

Rough total per clinic:

- With hosted APIs: about **$80–200/month**
- With a self-hosted GPU LLM: about **$400–1,000/month**. One GPU can be shared across several clinics' deployments to cut the cost, while keeping their data in separate databases.

What to charge the client (suggestion)

- One-time setup fee for the build, integration and testing.
- A monthly fee that covers usage plus maintenance and support, priced above your running costs.

8. Hosted LLM APIs: where to get them and what they cost

Hosted Qwen or GLM costs a fraction of a cent per call. The voice (TTS) is the biggest running cost.

Provider	Models	Approx. price	Data location
Groq	Qwen and other open models	about $0.10–0.80 per million tokens	US
OpenRouter	Qwen, GLM, many others (routes to other providers)	depends on the provider, often under $1 per million tokens	Varies
Together AI	Qwen, GLM, Llama	about $0.20–1.20 per million tokens	US
DeepInfra	Qwen, GLM	about $0.05–0.80 per million tokens	US

Provider	Models	Approx. price	Data location
Alibaba Cloud Model Studio	Qwen (official)	low, varies by model	Outside Canada

How to connect: sign up, create an API key, then set the LiveKit OpenAI plugin's base_url and api_key to that provider. A typical 3-minute call uses about 3,000–8,000 tokens, which is well under 1 cent.

Privacy note: all of these process data outside Canada. Use them for testing with fake data. For production, self-host the model in Canada (section 5, step 3), or get the clinic's privacy sign-off.

9. Why you need a database

The database is where every call is stored after it ends, and the front desk page reads from it.

Tables to create (PostgreSQL):

- **calls:** id, caller_name, caller_phone, started_at, duration, reason, urgency, summary, transcript, recording_url, status (New / In progress / Handled)
- **appointments_requests:** id, call_id, requested_date, service, booking_status
- **staff_users:** id, name, email, password_hash, role
- **audit_log:** who viewed or changed which record and when (needed for health privacy)

Without it, messages vanish when the call ends and there is no record for staff or audits. Host it in Canada (AWS RDS in ca-west-1), encrypted.

10. Front desk dashboard (web page for staff)

The dashboard is a secure web page where staff see every call the AI handled and act on it.

Features:

1. **Login:** staff accounts only, with strong passwords and optional two-factor sign-in. Sessions time out automatically.

2. **Call list:** caller name, phone number, time, reason, urgency badge and status, newest first, with filters for New, Urgent and Today.
3. **Call detail:** AI summary, full transcript, recording player (if consent was given), requested appointment and a call-back button.
4. **Actions:** mark as handled, add a staff note, assign to a staff member.
5. **Live updates:** new calls appear instantly through WebSockets, with a sound or browser notification.
6. **Alerts:** SMS or email to the front desk for urgent calls, without patient details in the text, just "New urgent call, check the dashboard".

How it connects:

1. The call ends, and the agent sends the summary to the backend API (POST /calls).
2. The backend saves the call to PostgreSQL and pushes a WebSocket event.
3. The dashboard receives the event and shows the new call.
4. Staff act on it in the dashboard, which calls the backend API, which updates the database and the audit log.

Stack: FastAPI backend (already used for tools), with React or Next.js for the dashboard. Host it on the same Canadian server over HTTPS. Cost: roughly $0–20/month extra on the existing server.

11. Privacy-safe voice (alternatives to ElevenLabs)

To keep patient voice data in Canada, self-host an open-source TTS on your Canadian server. Then no audio or text leaves the country.

Option	License	Quality	Speed	Hardware
Kokoro (82M)	Apache 2.0 (commercial OK)	Very natural	Very fast	CPU OK, GPU better
Piper	MIT (check each voice's license)	Good, slightly robotic	Extremely fast	CPU only
Chatterbox (Resemble AI)	MIT	Very natural, voice cloning	Medium	GPU
Orpheus TTS	Apache 2.0	Very expressive	Medium	GPU

Option	License	Quality	Speed	Hardware
XTTS v2 (Coqui)	Non-commercial license	Good	Medium	GPU, **avoid for client work**

Recommended: **Kokoro**, run through an OpenAI-compatible server (for example, Kokoro-FastAPI). Point the LiveKit OpenAI TTS plugin at it, so no code change is needed beyond the base_url.

Do the same for speech-to-text: self-host **faster-whisper** (Whisper, MIT license) in Canada instead of Deepgram.

Fully Canadian stack: Twilio number, self-hosted LiveKit, faster-whisper, Qwen on vLLM and Kokoro, all on one AWS ca-west-1 GPU server. Approximate cost: $300–800/month for the GPU server, which can be shared across clinics with separate databases.

If you keep a cloud voice vendor anyway: sign a data-processing agreement, turn on zero data retention, and record it in the clinic's Privacy Impact Assessment. Twilio still carries the call audio, so it belongs in the PIA too. Confirm the final setup with the clinic's privacy officer, since I'm not a lawyer.

12. Connecting the clinic's existing phone number

The clinic keeps its current number, and patients dial the same number as always. Pick one of three options once you know the clinic's phone provider.

Option	How it works	Extra cost	Setup time	Best when
1. Call forwarding	The clinic's provider forwards calls to a hidden Twilio/Telnyx number that is connected to LiveKit	about $1–2/month for the hidden number + per-minute rates	Same day	Landline or any provider; quickest start
2. Port the number	The number moves to Twilio/Telnyx and goes straight into LiveKit over a SIP trunk	Porting is usually free or a few dollars	1–3 weeks in Canada	You want full control long term

Option	How it works	Extra cost	Setup time	Best when
3. Direct SIP	The clinic's VoIP system (e.g. RingCentral, Telus Business Connect) sends calls over SIP straight to LiveKit	None for a number	1–3 days	The clinic already uses VoIP that supports SIP

Forwarding modes (option 1):

- **Always:** the AI answers every call.
- **Busy / no answer:** the AI answers only when staff can't pick up (for example, after 4 rings).
- **After hours:** the AI answers outside clinic hours, using a schedule on the provider's side.

Setup for option 1:

1. Buy one Canadian number on Twilio or Telnyx (not shown to patients).
2. Connect it to LiveKit with a SIP trunk and dispatch rule (section 5, step 6).
3. Ask the clinic's provider to set the forwarding to that number. For a landline this is often a star code, such as *72 for always on many Canadian lines; check with the provider.
4. Test by calling the clinic's real number.
5. In the agent, read the caller's original number from the forwarded call (the SIP From / caller ID) so the dashboard shows the real patient number.

Questions to ask the clinic:

- [ ] Who is the phone provider (Telus, Shaw/Rogers, Bell, RingCentral, other)?
- [ ] Is it a regular landline or internet-based VoIP?
- [ ] Should the AI take all calls, only missed calls, or only after-hours calls?
- [ ] Which number should urgent calls be transferred to?
- [ ] Who has admin access to the phone account?

13. Timeline (rough)

Week	Work
1	Accounts, basic agent in the Playground, Qwen connected
2	Tools, booking integration, clinic config
3	Phone number + SIP, testing from India
4	Canadian deployment, privacy setup, soft launch

14. Checklist before go-live

- [ ] AI disclosure and recording notice in the greeting
- [ ] Emergency handling (911) and human transfer tested
- [ ] No medical advice in the prompt rules
- [ ] Data stored in Canada, encrypted
- [ ] Agreements signed with the clinic and vendors
- [ ] Latency under about 1.5 seconds
- [ ] Call logs and staff notifications working
- [ ] Monitoring and alerts on
- [ ] Fallback if the AI is down (forward the call to the clinic line or voicemail)